Petal Ping

Product Brief · Final · AI Builder Space Proseminar
Elle Brauchle, MBA Candidate at MIT Sloan '28

October 19, 2026

Petal Ping is an integration tool that pulls real menstrual cycle and symptom data from a health app to deliver tailored, research-backed notifications and trend insights — built solo as a free, personalized alternative to tracking tools that are either too limited or too costly.

SECTION 1

Basics

Project: Petal Ping | **Creator:** Elle Brauchle (solo project) | **Type:** integration tool

Status: shipped and in daily use. Every must-have from the original brief is built and running. The unattended daily job — the one piece outstanding at Sprint 1 — now sends on schedule without the laptop open. The companion app exists behind a password, feedback is collected per notification, and that feedback has already changed what the app says.

The binding constraint is no longer engineering. It is data volume: two complete cycles on record, which holds prediction confidence at "low" until roughly four.

SECTION 2

The Idea & The User

Unchanged from Sprint 1, and worth restating because the build did not drift from it.

Petal Ping connects to an existing health app to track menstrual cycles and symptoms, and delivers tailored push notifications — heads-up reminders for upcoming cycles, research-backed health tips, and eventually alerts when trends emerge. A companion app displays this information. It begins as a personal tool for its creator, with potential to expand to anyone managing periods, especially those with challenging or irregular cycles.

What the build clarified about the user: the hard problem is not prediction, it is **tone and frequency**. A cycle app that is wrong is annoying; one that is right but says it badly, or says it too often, gets muted — which is the same as being deleted. Sprint 2 treated that as an engineering problem with a measurable answer rather than a matter of taste (Section 5).

SECTION 3

Existing Options

Unchanged. Apple Health's native tracking is basic; Flo and Natural Cycles are paid and closed. Petal Ping was built from scratch to learn to build software, because the tool directly benefits its creator, and to avoid a recurring subscription. All three still hold, and the running cost is confirmed at **$0** — Vercel free tier, no Apple Developer Program, no subscription.

Clue remains a data feed only. Everything downstream is original. One addition this sprint: Clue's **published articles** are now a cited source for tip content, paraphrased with attribution rather than reproduced, alongside ACOG, NIH, the Office on Women's Health, Cochrane and the Sleep Foundation.

SECTION 4

Scope: What Shipped

The smallest useful version was "real data flowing through to tailored notifications." That is done. Final status against the original must-haves:

Must-haves — all complete

- **Pull real cycle/symptom data — done.** `ingest.py` reads the newest Clue export, normalizes it, and computes a prediction. Current store: three period episodes, two complete cycles, and 31 symptom records across 10 types.
- **Predictive heads-up notifications — done.** A daily scheduled job on Vercel reads the stored prediction, asks a rules engine what is worth saying, and sends at most one notification. Verified sending unattended. Six rules, ordered most urgent to most routine. Predictions are stated as a window with explicit confidence, never a single confident date.
- **Research-backed tips — done.** A curated library, now 16 tips, each carrying its source. No generated text, so no invented health claim can reach the lock screen.
- **Companion app — done** (promoted from "out of scope"). A home-screen PWA showing days until the next window, the window itself, stated confidence, cycle day, phase, and a preview of today's notification. Behind a password: scrypt-hashed, signed session cookie, lockout on repeated failures.
- **Per-notification feedback — done** (was an open question). Thumbs up/down with an optional free-text note, stored server-side.

Still out of scope

- **Trend alerts** — deferred on data volume, not effort. Needs the same pattern across two or more complete cycles of symptom logging; realistically December.
- **Export for medical appointments** — not built.
- **Flow-intensity notification** — the hypothesis was tested and failed (see Section 6).

SECTION 5

What Does Success Look Like?

The Sprint 1 definition was "pulls real data and sends at least one real predictive notification and one research-backed tip." **Both are met**, unattended, on schedule.

Because success at that bar was reached early, this sprint set a harder one: **does the thing improve itself from use?** The loop now runs end to end:

- Five versions of the same notification were sent in different voices — supportive, clipped, clinical, coaching, playful — each rated separately in the app.
- Four were rated useful; the clipped one was the only rejection. The written notes asked for humor ("keeps a daunting thing lighthearted"), an actionable closing clause ("I like the 'use it'"), and visible sourcing ("data backed").
- All 16 tips and four rule bodies were rewritten against those notes, and committed with the feedback that justified the change.

That is a complete build–measure–learn cycle with the evidence preserved in version control.

What success cannot yet mean: predictive accuracy. Cycles run monthly, so with two complete cycles the honest statement is a four-day window at low confidence. A two-week daily-notification trial is now running (auto-expiring November 2) to answer the frequency question with lived experience rather than a guess.

SECTION 6

Open Questions, Risks, and What Was Learned

Closed since Sprint 1

- Scheduled job verified end to end — sends unattended.
- Server-side storage solved (private blob store; only the prediction leaves the laptop).
- Per-notification feedback shipped, with notes.
- Tips: curated library confirmed over retrieval-augmented generation. Sixteen cited items covers the content need without machinery or hallucination risk.

Still open

- **Ideal notification frequency.** The daily trial through November 2 is the experiment.
- **Predictive accuracy.** Unanswerable until roughly four complete cycles.
- **Symptom-based features.** 31 records now exist, but inside a single cycle. Threshold for revisiting is explicit: the same pattern across two complete cycles.

Risks accepted, with mitigations

- **Manual ingest.** A scheduled job was built and removed after it never once succeeded — macOS blocks background processes from reading protected folders, and the fix would have meant granting a general-purpose interpreter access to the entire machine. Ingest is now run on request. The failure mode is silent drift against a stale prediction; mitigated by a staleness banner in the app after 21 days.
- **Single point of failure in push.** If the subscription expires, notifications stop quietly. The app detects this and prompts to re-register.
- **Privacy.** Menstrual data carries real legal exposure. Only the prediction conclusion is uploaded; the full history never leaves the laptop. The repository is public, and a pre-commit hook blocks any commit containing health data or secrets — it has already caught real mistakes, including a `.gitignore` whose trailing comments silently disabled it.

What was learned, in order of usefulness

- **A falsified hypothesis costs one paragraph if it was marked provisional.** A flow-intensity pattern held across two cycles and looked worth building on. It was recorded as provisional, tested against the third cycle, and failed. Because nothing had been built on it, the cost was a documentation entry rather than a broken feature.
- **Enforce constraints with tooling, not intention.** The decisions document claimed on its first page to contain no personal health data. It contained three cycles of flow values. A filename check would not have caught it; a content check did.

- **Test the risky assumption before building anything around it.** iOS Web Push was the likeliest thing to fail, so it was spiked first as a throwaway endpoint with no UI. It worked on the first attempt, which retired the largest risk before a line of interface existed.
- **Deterministic beats clever when the output is health advice.** Feedback could have driven a model that writes notifications. Instead it drives hand edits to a rules file, where every change is reviewable in git. The risk being avoided — plausible-sounding invented health claims with citations that do not support them — is specific, not hypothetical.

Full decision log, including all thirteen decisions with their rejected alternatives, is in `Ongoing Decisions Doc.md` *in the project repository.*